

Sentencias de decisión y selección en Python

1. Sentencias de decisión (if)

- Controlan el **flujo del programa** verificando si una condición es **verdadera o falsa**.
- Si la condición se cumple, se ejecuta el bloque de código indentado.

Ejemplo – Envío gratuito con if:

```
if order_total >= 50:  
    print("Congratulations! You qualify for free shipping.")
```

Si el total del pedido es mayor o igual a 50, se muestra el mensaje. Si no, no ocurre nada.

2. Sentencias de selección (if-else)

- Ofrecen **dos caminos**:
 - Uno si la condición es **verdadera**.
 - Otro si es **falsa**.

Ejemplo – Envío gratuito con if-else:

```
if order_total >= 50:  
    print("Congratulations! You qualify for free shipping.")  
else:  
    print("Add more items to your cart to reach the free shipping  
threshold!")
```

Si no alcanza los 50, se anima al cliente a añadir más productos.

3. Sentencias múltiples (if-elif-else)

- Permiten evaluar **varias condiciones secuenciales de arriba a abajo**.
- Python ejecuta el **primer bloque verdadero** y luego se detiene.

Ejemplo – Niveles de afiliación con elif:

```
if points >= 1000:  
    print("You've achieved Platinum status! Enjoy exclusive  
benefits.")  
elif points >= 500:  
    print("You're a Gold member! Thank you for your loyalty.")  
elif points >= 100:  
    print("Welcome to the Silver tier! Start earning rewards.")  
else:  
    print("You're a Bronze member. Keep shopping to earn more  
points!")
```

Evalúa de mayor a menor rango de puntos, asignando el nivel adecuado al cliente.

Conclusión

- **if**: decisión binaria (ejecutar o no ejecutar).
- **if-else**: añade una **alternativa** cuando la condición no se cumple.
- **if-elif-else**: gestiona **múltiples escenarios** de forma clara y eficiente.

Condiciones Anidadas y Lógica Booleana en Python

1. Condiciones Anidadas

- Son **sentencias if dentro de otras if/else**.
- Funcionan como un **árbol de decisiones jerárquico**.

- Se usan cuando una decisión depende de otra condición previa.

Ejemplo básico:

```
if age >= 18:
    if has_id:
        print("Acceso permitido")
    else:
        print("Necesitas una identificación")
else:
    print("Debes ser mayor de edad")
```

2. Lógica Booleana

- Se basa en operadores lógicos:
 - **and** → ambas condiciones deben cumplirse.
 - **or** → basta con que una condición se cumpla.
 - **not** → invierte el valor de la condición.

Ejemplo con lógica booleana:

```
is_student = True
age = 65

if age >= 65 and is_student:
    print("Descuento especial para estudiantes mayores")
```

Requiere que **ambas condiciones** sean verdaderas.

3. Importancia de Definir Variables

- Antes de usar una condición, las variables deben estar **inicializadas** (ej. `is_student = True`).
- Si no lo están, Python generará un **NameError**.

4. Aplicaciones Reales

Las condiciones anidadas y la lógica booleana impulsan aplicaciones del mundo real:

- **Vehículos autónomos** → toman decisiones en tiempo real sobre detenerse, avanzar o girar.
- **Sistemas antifraude** → detectan patrones sospechosos en transacciones.
- **Plataformas de aprendizaje** → ajustan dificultad y recomendaciones según el progreso del alumno.
- **Domótica inteligente** → automatizan temperatura, seguridad y energía en el hogar.
- **Videojuegos y simulaciones** → crean mundos dinámicos con PNJ que reaccionan distinto según las acciones del jugador.

En conclusión: **las condiciones anidadas y la lógica booleana permiten modelar decisiones complejas en Python**, desde simples validaciones hasta sistemas inteligentes como coches autónomos o plataformas educativas adaptativas.

Complejidad y Mantenimiento en Sentencias de Decisión

1. Posibles problemas

- Un **uso excesivo de if/else anidados** puede generar código:
 - Difícil de leer.
 - Complicado de mantener y depurar.
 - Propenso a errores e ineficiencias.

2. Buenas prácticas para mejorar el código

- **Modularización** → dividir en funciones pequeñas y manejables.
- **Nombres descriptivos** para variables.
- **Comentarios claros** para explicar la lógica.
- Considerar **estructuras alternativas** (ej. diccionarios o switch en otros lenguajes).

3. Optimización de las decisiones

- **Diccionarios en lugar de elif múltiples:**
 - Hacen el código más limpio y fácil de actualizar.
 - Ejemplo: asignar tarifas de envío a códigos de país.
- **Uso de librerías como NumPy o Pandas:**
 - Realizan operaciones condicionales de forma más **eficiente y rápida**.
 - Reducen tiempo y esfuerzo de desarrollo.

4. Conclusión

- Las **sentencias de decisión** son la base de programas dinámicos.
- Siguiendo buenas prácticas y explorando alternativas, se puede lograr código:
 - **Eficiente**
 - **Escalable**
 - **Fácil de mantener**

Bucles y Sentencias Condicionales en Python

La programación necesita **repetir tareas** y **tomar decisiones**. Para esto se usan:

- **Bucles** → repiten instrucciones automáticamente.
- **Sentencias condicionales** → permiten tomar decisiones según condiciones.

1. Bucles: El poder de la repetición

- Evitan escribir código repetitivo.
- Permiten ejecutar instrucciones varias veces de forma eficiente.
- Dos tipos principales en Python: **for** y **while**.

a) Bucle **for** – Iterar sobre secuencias

- Recorre listas, tuplas, cadenas o rangos de números.
- Útil cuando sabemos **cuántas veces** ejecutar el bucle.

Ejemplo – Imprimir los números del 1 al 10:

```
for i in range(1, 11):  
    print(i)
```

`range(1, 11)` genera los números del 1 al 10, y `i` toma cada valor.

Aplicaciones: procesar listas de nombres, calcular precios en un carrito, recorrer caracteres de una cadena.

b) Bucle **while** – Ejecución basada en condiciones

- Repite un bloque de código **mientras una condición sea verdadera**.
- Útil cuando no sabemos cuántas iteraciones habrá.

Ejemplo – Pedir entrada válida al usuario:

```
valid_input = False

while not valid_input:
    user_input = int(input("Please enter a number greater than 0:
"))
    if user_input > 0:
        valid_input = True
    else:
        print("Invalid input. Please try again.")
```

El bucle se repite hasta que el usuario introduce un número mayor que 0.

Aplicaciones: juegos, simulaciones, procesamiento en tiempo real.

Conclusión

- Los **bucles** permiten eficiencia en tareas repetitivas.
- El **for** se usa para secuencias conocidas.
- El **while** se usa cuando la condición de parada depende de la lógica del programa.

1. Ventajas del uso de bucles

- **Reducen duplicación de código:** evitan repetir líneas innecesarias, lo que facilita el mantenimiento.
- **Mejoran legibilidad y depuración:** es más fácil encontrar y corregir errores.
- **Permiten recorrer estructuras de datos** (listas, diccionarios, etc.) de manera eficiente.

- **Automatizan tareas repetitivas**, ahorrando tiempo y permitiendo que el programador se concentre en problemas más complejos.

2. Sentencias condicionales: Toma de decisiones

- Permiten que un programa **ejecute diferentes bloques de código** según si una condición es **verdadera o falsa**.
- Añaden **dinamismo** a los programas, haciéndolos responder a diferentes entradas o situaciones.
- La sentencia más común en Python es **if-else**.

Ejemplo: if-else en un juego

```
player_score = 80

if player_score > 100:
    print("Congratulations! You scored over 100 points.")
else:
    print("Keep trying to beat your high score!")
```

Resultado:

```
Keep trying to beat your high score!
```

Aquí, el programa revisa la condición (`player_score > 100`) y ejecuta el bloque correspondiente.

3. Importancia

- Las condicionales no solo muestran mensajes, sino que también pueden:
 - **Controlar el flujo del programa.**
 - **Determinar qué funciones ejecutar.**

- **Procesar diferentes datos según las circunstancias.**

Importancia de las Sentencias Condicionales y los Bucles

1. Importancia de las sentencias condicionales

- Controlan el **flujo del programa**, ejecutando bloques de código según condiciones.
- Mejoran la **interactividad** y la **adaptabilidad** del software.
- Implementan **lógica y reglas**: validación de datos, gestión de errores y toma de decisiones basadas en análisis.
- Garantizan **seguridad** (ejemplo: comprobar contraseñas o correos válidos).

2. Aplicaciones reales

- **Análisis de datos**: bucles recorren grandes conjuntos de datos y condicionales filtran información (ej. clientes que gastan más de cierto valor).
- **Desarrollo web**: bucles generan contenido dinámico y condicionales controlan interacciones como formularios o login.
- **Desarrollo de juegos**: bucles gestionan el ciclo del juego y condicionales reaccionan a acciones del jugador (movimientos, colisiones, objetos).
- **Aprendizaje automático**: bucles entrenan modelos y condicionales ajustan parámetros según métricas de rendimiento.

3. Problemas potenciales

- **Complejidad innecesaria**: bucles anidados o condiciones confusas dificultan la lectura.

- **Bucle infinito:** si la condición nunca se vuelve falsa, el programa puede bloquearse.

4. Buenas prácticas

- Mantener bucles **concisos y organizados**.
- Usar **condiciones claras y significativas**.
- Considerar **alternativas elegantes** como listas por comprensión o lambdas.
- Evitar bucles infinitos mediante condiciones bien definidas.

5. Conclusión

Los **bucles** y las **sentencias condicionales** son la base de la programación:

- Automatizan tareas repetitivas.
- Permiten decisiones dinámicas.
- Hacen que los programas sean más **eficientes, interactivos y fáciles de mantener**.

Dominar estas herramientas es clave para crear software **funcional, elegante y escalable**.

Desafío: Adivina el número secreto

1. Objetivo:

Crear un juego en el que el **ordenador intente adivinar un número secreto** que el programador define (entre 1 y 10).

2. Pasos principales:

- Definir una variable `secret_number` con un valor fijo (ej. 5).
- Inicializar una variable `guess = 0`.

- Usar un **bucle while** que siga ejecutándose mientras `guess` no sea igual a `secret_number`.
- Dentro del bucle, generar un número aleatorio con `random.randint(1, 10)` y mostrar la suposición.
- Cuando acierte, mostrar un mensaje confirmando el número correcto.

```
import random
```

```
secret_number = 5    # Número secreto definido manualmente  
guess = 0            # Inicializamos la suposición
```

```
while guess != secret_number:    # Mientras no adivine  
    guess = random.randint(1, 10) # Genera número aleatorio  
    print("Guessing:", guess)
```

```
print("I guessed the right number! It was", secret_number)
```

Feedback: Nice work! Your code correctly implements the guessing game.

Your output:

```
Guessing: 7  
Guessing: 1  
Guessing: 4  
Guessing: 10  
Guessing: 10  
Guessing: 6  
Guessing: 4  
Guessing: 2  
Guessing: 10  
Guessing: 2  
Guessing: 5  
I guessed the right number! It was 5
```

Este programa ejecutará intentos aleatorios hasta adivinar el número secreto.

Sentencias condicionales en Python

1. ¿Qué son las sentencias condicionales?

- Son los **guardianes de la lógica** en Python.
- Evalúan condiciones y ejecutan diferentes bloques de código según si son **verdaderas o falsas**.
- Permiten que los programas sean **dinámicos y adaptables**, como una historia de “elige tu propia aventura”.

2. Principales sentencias condicionales

- **if** → Comprueba una condición. Si es verdadera, ejecuta un bloque de código.
Ejemplo: Activar una bonificación si las ventas superan cierto umbral.
- **else** → Se usa junto a **if** para manejar la situación en que la condición es falsa.
Ejemplo: En un login, muestra un error si las credenciales no son válidas.
- **elif (else if)** → Permite comprobar varias condiciones en secuencia.
Ejemplo: Calcular tarifas de envío diferentes según el rango de precio en un carrito de compras.

3. Importancia de las condicionales

- Son la **columna vertebral** de los programas dinámicos.
- Permiten manejar múltiples escenarios y responder a entradas del usuario, datos o eventos externos.
- Ayudan a crear **árboles de decisión** complejos, validación de datos y gestión de errores.
- Facilitan experiencias **personalizadas** para los usuarios.

4. Aplicaciones reales

- **Juegos** → Controlan respuestas a las acciones del jugador (ejemplo: perder vida o curarse).
- **Validación de datos** → Comprobar contraseñas seguras o correos válidos en formularios.
- **Manejo de errores** → Detectar si falta un archivo y mostrar un mensaje en lugar de que el programa falle.
- **Interactividad personalizada** → Adaptar resultados según preferencias o comportamientos de los usuarios.

En conclusión, las sentencias condicionales en Python son esenciales para que el código **tome decisiones inteligentes**, maneje distintos escenarios y cree aplicaciones más útiles, dinámicas y seguras.

Diálogo

El problema que resolveremos es el siguiente:

Estás creando un programa para determinar el precio de las entradas de cine, donde el precio de la entrada depende de la edad del cliente. Asume que la edad es un número entero:

- Niños (menores de 12 años): \$8
- Adultos (12-64 años): \$12
- Mayores (65 años o más): \$10

```
## diálogo
```

```
edad = 78
```

```
if edad < 12:  
    print("El precio de tu entrada es de $8.")  
elif edad >= 12 and edad <= 64:  
    print("El precio de tu entrada es de $12.")  
elif edad >= 65:
```

```
print("El precio de tu entrada es de $10.")  
El precio de tu entrada es de $10.
```

```
## diálogo  
edad = int(input("Introduce tu edad: ")) ## otra forma
```

```
if edad < 12:  
    print("El precio de tu entrada es de $8.")  
elif edad >= 12 and edad <= 64:  
    print("El precio de tu entrada es de $12.")  
elif edad >= 65:  
    print("El precio de tu entrada es de $10.")
```

Repetición de acciones: Bucles For y While

En Python, los bucles permiten **repetir acciones de manera automática**, mejorando la eficiencia y evitando la duplicación de código. Son fundamentales para manejar datos, realizar cálculos repetitivos y automatizar tareas.

Tipos de bucles

1. Bucles For

Se utilizan cuando se conoce el número de iteraciones o se quiere recorrer una **secuencia** (lista, tupla, cadena, rango).



Ejemplo básico:

```
for i in range(1, 11): # Imprime del 1 al 10  
    print(i)
```

Formas de usar `range()`:

`range(stop)` → 0 (por defecto) hasta stop - 1

```
for i in range(5):  
    print(i) # 0,1,2,3,4
```

•

`range(start, stop)` → start hasta stop-1

```
for i in range(1, 11):  
    print(i) # 1,2,...,10
```

•

`range(start, stop, step)` → start hasta stop-1 con incremento step

```
for i in range(1, 10, 2):  
    print(i) # 1,3,5,7,9
```

```
for i in range(10, 5, -1):  
    print(i) # 10,9,8,7,6
```

•

2. Bucles While

Se ejecutan **mientras se cumpla una condición**, útil cuando el número de iteraciones no se conoce de antemano.

```
while condition:  
    # Code block to be executed repeatedly
```

Ejemplo básico:

```
number = 1  
  
while number <= 10:  
    print(number)  
    number = number + 1
```

Ejemplo de menú interactivo:

```
option = 0

while option != 4:

    print("1. Perform action 1")

    print("2. Perform action 2")

    print("3. Perform action 3")

    print("4. Quit")

    option = int(input("Choose an option: "))
```

Diferencias clave

- **for**: iteración sobre secuencias o rango conocido.
- **while**: se ejecuta hasta que una condición deja de cumplirse, útil para escenarios dinámicos (juegos, validación de entrada).
- Los bucles **for** son excelentes para iterar sobre secuencias conocidas, como listas o rangos definidos. Por otro lado, los bucles **while** son más adecuados cuando el número de repeticiones no se conoce de antemano y el bucle debe continuar hasta que se cumpla una condición específica.

Aplicaciones comunes

- Automatización de tareas repetitivas.
- Validación de datos de usuario hasta recibir un formato correcto.
- Bucles de juego que continúan hasta alcanzar condiciones específicas (puntos o vidas).

Listas y bucles en Python

Listas

Una **lista** es una estructura de datos que almacena una colección ordenada de elementos.

- Se definen usando corchetes `[]` y separando los elementos con comas.
- Los elementos pueden ser de cualquier tipo (números, cadenas, objetos, etc.).

Ejemplo:

```
fruits = ["apple", "banana", "cherry"]
```

Acceso a elementos

Cada elemento tiene un **índice** empezando en 0.

```
first_fruit = fruits[0] # "apple"
```

Funciones útiles

- `len(lista)` devuelve la cantidad de elementos en la lista:

```
fruit_length = len(fruits)

print(fruit_length) # 3
```

- `append()` añade un elemento al final de la lista:

```
fruits.append("date")

print(fruits) # ["apple", "banana", "cherry", "date"]
```

Recorrer listas con bucles

Bucle **for each** (más limpio y directo)

```
students = ["Alice", "Bob", "Charlie"]

for student in students:
```

```
print("Hello,", student)
```

Salida:

```
Hello, Alice
```

```
Hello, Bob
```

```
Hello, Charlie
```

Este método **itera automáticamente sobre cada elemento**, asignándolo a una variable temporal (`student`) y ejecutando el bloque de código.

Bucle `for` con `range()` (más manual)

```
for i in range(0, len(students)):  
    print("Hello,", students[i])
```

Funciona igual, pero requiere **gestionar manualmente los índices**.

Aplicaciones prácticas

- **Procesamiento de imágenes:** recorrer píxeles de una imagen para modificar colores o aplicar filtros.
- **Web scraping:** iterar sobre listas de URLs para extraer información de varias páginas.

Aprovechar los bucles para la iteración

Bucles `while`

- Se utilizan cuando **no se conoce de antemano el número de repeticiones**.
- Ejecutan un bloque de código mientras se cumpla una condición.

Ejemplo: simular tiradas de dados hasta obtener un 6

```
import random
```

```
roll = 0

while roll != 6:

    roll = random.randint(1, 6)

    print("You rolled a", roll)
```

Contar y procesar datos con bucles **for**

- Ideales para **iterar sobre un rango de valores conocido**.
- Se pueden usar para sumar números, contar elementos, etc.

Ejemplo: suma de números del 1 al 100

```
total = 0

for number in range(1, 101):

    total += number

    print("The sum is:", total)
```

Bucles **while** con condiciones complejas

- Permiten filtrar o procesar datos según condiciones específicas.

Ejemplo: imprimir números pares de una lista

```
numbers = [1, 2, 3, 4, 5]

index = 0

while index < len(numbers):

    if numbers[index] % 2 == 0:
```

```
        print(numbers[index])

    index += 1
```

Salida:

2

4

Bucles anidados: iteración avanzada

- Se usan para **operaciones más complejas** con estructuras de datos múltiples.
- El bucle exterior itera sobre un conjunto, y el interior realiza iteraciones adicionales para cada elemento.

Ejemplo: tabla de multiplicar

```
for i in range(1, 11):

    for j in range(1, 11):

        print(i, "*", j, "=", i * j, end="\t") # Tabulación en
        lugar de nueva línea

    print() # Salto de línea tras cada fila
```

Salida parcial:

```
1 * 1 = 1    1 * 2 = 2    1 * 3 = 3 ... 1 * 10 = 10
2 * 1 = 2    2 * 2 = 4    2 * 3 = 6 ... 2 * 10 = 20
...
10 * 1 = 10  10 * 2 = 20  10 * 3 = 30 ... 10 * 10 = 100
```

Aplicaciones prácticas

- **Bucles `while`:** juegos, validación de entradas de usuario, simulaciones hasta que ocurra un evento.
- **Bucles `for`:** cálculos matemáticos, procesamiento de listas y rangos de números.
- **Bucles anidados:** tablas, matrices, imágenes, operaciones sobre estructuras de datos multidimensionales.

Desafío

desafío utilizando un **bucle `for`** y una **sentencia `if`** para comprobar la divisibilidad entre 3 y 4:

```
# Valor máximo
max_value = 50
number = 0 # opcional porque ya empieza a contar desde 0

# Bucle for para recorrer los números desde 0 hasta max_value inclusive
for number in range(max_value + 1):
    # Comprobar si el número es divisible por 3 y por 4
    if number % 3 == 0 and number % 4 == 0:
        print(number)
```

Explicación:

1. `range(max_value + 1)` genera números de 0 a 50 inclusive.
2. Dentro del bucle, la condición `number % 3 == 0 and number % 4 == 0` verifica que el número actual sea divisible por **3 y 4**.
3. Si la condición es verdadera, se imprime el número.

Salida esperada:

0

12
24
36
48

Uso de bucles en Python

Los bucles son esenciales para escribir código eficiente, legible y adaptable en Python. Permiten automatizar tareas repetitivas, manejar datos de forma efectiva y mejorar la calidad general de los programas. Para aprovecharlos al máximo:

- **Elige el bucle adecuado:** Usa `for` cuando el número de iteraciones sea conocido y `while` cuando dependa de una condición.
- **Sé conciso:** Evita cuerpos de bucle demasiado complejos; divide tareas en funciones para mantener el código modular y fácil de entender.
- **Usa nombres significativos:** Variables descriptivas facilitan la lectura y el mantenimiento, especialmente en colaboración.
- **Mantén sangrías correctas:** La indentación clara es clave en Python para reflejar la estructura del bucle.
- **Optimiza la eficiencia:** Minimiza cálculos dentro del bucle; considera comprensiones de lista o expresiones generadoras cuando sea posible.

En conjunto, los bucles `for` y `while` son pilares de la programación en Python, permitiendo iteraciones elegantes y soluciones eficaces a problemas complejos mediante buenas prácticas y técnicas de optimización. Los bucles `for` son excelentes para iterar sobre secuencias conocidas, como listas o rangos definidos. Por otro lado, los bucles `while` son más adecuados cuando el número de repeticiones no se conoce de antemano y el bucle debe continuar hasta que se cumpla una condición específica.

Flujo de control en Python

El **flujo de control** en Python determina qué instrucciones se ejecutan, en qué orden y bajo qué condiciones, funcionando como un “director de orquesta” para tu código. Es esencial para crear programas adaptables, dinámicos y seguros.

Sentencias condicionales: Tomar decisiones

Las sentencias condicionales (`if`, `elif`, `else`) permiten que el código evalúe condiciones y ejecute distintos bloques según si la condición es verdadera o falsa.

Ejemplo básico **if-else**:

```
temperature = 22 # Temperatura de ejemplo

if temperature < 20:

    print("It might be cold! You might need a jacket.")
else:

    print("It's warm! Enjoy the sunshine.")
```

Salida:

It's warm! Enjoy the sunshine.

Ejemplo con **elif** para múltiples condiciones:

```
temperature = 15

if temperature < 10:

    print("Wear a jacket! It's cold out there.")
elif 10 <= temperature < 20:

    print("A light sweater should be fine. It's a bit chilly.")
elif 20 <= temperature < 30:

    print("Enjoy the pleasant weather! No need for extra layers.")
else:

    print("It's hot! Stay hydrated and wear sunscreen.")
```

Salida:

A light sweater should be fine. It's a bit chilly.

Bucles: Automatizar tareas repetitivas

Los bucles permiten ejecutar un bloque de código varias veces:

- **Bucle `for`:** Ideal cuando se conoce el número de iteraciones.

```
# Imprimir números del 1 al 5

for i in range(1, 6):

    print(i)
```

Salida:

```
1
2
3
4
5
```

- **Bucle `while`:** Continúa mientras se cumpla una condición, útil cuando el número de iteraciones no es conocido.

```
# Solicitar entrada hasta que el usuario escriba "quit"

user_input = ""

while user_input != "quit":

    user_input = input("Enter something (or 'quit' to exit): ")

    print("You entered:", user_input)
```

Conclusión

El flujo de control, mediante **condicionales y bucles**, permite que los programas tomen decisiones inteligentes y gestionen tareas repetitivas. Esto hace que los programas sean más **flexibles, interactivos y potentes**, y es una base esencial para cualquier desarrollador Python.

Aplicaciones reales del flujo de control en Python

El **flujo de control** es esencial en casi todas las aplicaciones informáticas, permitiendo que los programas tomen decisiones y repitan acciones de manera inteligente mediante **sentencias condicionales y bucles**.

Aplicaciones destacadas:

1. Desarrollo de juegos

- **Condicionales:** Verifican eventos como colisiones del jugador o victorias.
- **Bucles:** Actualizan continuamente posiciones, cronómetros y estados del juego para una experiencia en tiempo real.

2. Desarrollo web

- **Condicionales:** Determinan qué contenido mostrar según el estado del usuario (ej. inicio de sesión).
- **Bucles:** Generan dinámicamente listas de productos, menús o recomendaciones desde bases de datos.

3. Análisis de datos

- **Bucles:** Iteran sobre grandes conjuntos de datos para cálculos estadísticos o identificación de patrones.
- **Condicionales:** Filtran información irrelevante o eliminan valores atípicos para mejorar el análisis.

4. Automatización

- **Bucles:** Monitorean continuamente tareas repetitivas como bandejas de entrada de correo electrónico.
- **Condicionales:** Deciden acciones según condiciones específicas, como enviar recordatorios o alertas.

Recomendaciones

- Comenzar con ejemplos sencillos y practicar gradualmente.
- Experimentar y aprender de los errores es clave para dominar el flujo de control.
- Con estas herramientas, es posible crear programas dinámicos, interactivos y eficientes, desde **juegos y aplicaciones web** hasta **procesos de automatización**.

Desafío de programación: Número impar

Su tarea:

Escribe un programa en Python que tome una lista de números e itere por cada número y compruebe si es par o impar. Utiliza la lista de números proporcionada como punto de partida.

- Iterar sobre la lista de números: Utilice un bucle for para iterar sobre la lista proporcionada (`for number in numbers:`). Utilice el nombre de variable `number` para la variable.
- Comprobar par/impar: Utilice una sentencia condicional para determinar si cada número es par o impar.
- Mostrar salida: Imprima un mensaje indicando si los números son pares o impares.

Consejos:

- Puede utilizar el operador de módulo (%) para comprobar si un número es divisible por 2.
- Si el resto de la división por 2 es 0, el número es par.
- En caso contrario, es impar.

`## Desafío: escribir si los números de la variable number son pares o impares usando for number in numbers`

```
numbers = [3, 9, 1, 10, 5, 2, 8]
```

```
number = 0
```

```
for number in numbers:
    if number %2 == 0:
        print(number, 'es par')
    else:
        print(number, 'es impar')
```

3 es impar

9 es impar

1 es impar

10 es par

5 es impar

2 es par

8 es par

Diálogo

Diálogo cuenta regresiva del 10 al 0 con mensaje especial en el 5 usando for y condicionales

```
for cuenta_regresiva in range(10, -1, -1):
    if cuenta_regresiva == 5:
        print(cuenta_regresiva, 'Has llegado a la mitad')
    else:
        print(cuenta_regresiva)
```

10

9

8

7

6

5 Has llegado a la mitad

4

3

2

1

0

Seguimiento de la ejecución en Python

1. ¿Qué es el rastreo?

- Proceso de seguir la ejecución del código paso a paso.
- Sirve para **depurar errores**, comprender la lógica de programas complejos y optimizar el código.
- Ventajas: claridad, diagnóstico de problemas y mejor colaboración.

2. Componentes clave en la ejecución de Python

- **Script**: conjunto de instrucciones escritas.
- **Intérprete**: ejecuta el código línea por línea, traduciéndolo a instrucciones de máquina.
- **Pila de llamadas**: registra funciones activas y puntos de retorno (útil para entender flujos y detectar errores como recursión infinita).
- **Variables**: almacenan datos que cambian durante la ejecución.

3. Herramienta principal de rastreo

- La instrucción `print()`.
- Permite:
 - Inspeccionar valores de variables.
 - Verificar el orden de ejecución.
 - Detectar resultados inesperados.

4. Ejemplos prácticos

- **Función simple (`greet`):**
 - Se pasa un argumento ("Alice"), se crea un mensaje y se retorna.
 - El `print` muestra el resultado.
- **Funciones anidadas (externa + interna):**
 - La externa calcula $y = x + 5$.
 - La interna usa y y calcula $z = y * 2$.
 - Se imprimen los valores de y y z .
 - Muestra cómo funciona el **alcance de variables** y el orden de ejecución.

5. Conclusiones

- El rastreo ayuda a entender el flujo del programa, el alcance de las variables y la interacción entre funciones.
- Es fundamental para depuración, aprendizaje y mejora del código en proyectos pequeños o grandes.

Errores comunes en la ejecución de código en Python y cómo evitarlos

Python es poderoso y sencillo, pero tiene reglas que, si no se respetan, generan errores. Aquí están los más comunes:

1. Errores de sangría (IndentationError)

Python usa **sangría** (espacios/tabulaciones) para definir bloques de código.

Un error común es alinear mal un bloque `if-else`.

Error:

```
x = 0

if x > 5:
    print("x is greater than 5")
else: # Incorrect indentation
    print("x is less than or equal to 5")
```

Error:

```
IndentationError: unindent does not match any outer indentation level
```

Corrección:

```
x = 0

if x > 5:
    print("x is greater than 5")
else:
    print("x is less than or equal to 5")
```

2. Ámbito de las variables (NameError)

Las variables solo existen dentro de su **ámbito** (donde fueron definidas).

✗Error:

```
flag = False

if flag:

    x = 10

    print(x)  # No problem here


print(x)  # Error si flag = False
```

Error:

```
NameError: name 'x' is not defined
```

Corrección:

```
flag = False

x = None  # Definir fuera del if

if flag:

    x = 10

    print(x)


print(x)

# otra forma
```

```
flag = True
```

```
if flag:  
    x = 10  
    print(x)
```

```
print(x)
```

3. Valores inmutables (TypeError con strings)

Las **cadenas (str)** son **inmutables**, no se pueden modificar directamente.

✗Error:

```
value = "Hello"  
value[0] = 'h'  
print(value)
```

❑ Error:

```
TypeError: 'str' object does not support item assignment
```

✓Corrección:

```
value = "Hello"  
new_value = "h" + value[1:]  
print(new_value) # Output: hello # habrá más formas en el futuro
```


4. Reasignación de variables sin querer

Reutilizar un nombre de variable cambia su valor, a veces de forma no deseada.

✗Error:

```
x = 5

if True:

    x = 10  # Reutilizado accidentalmente

    print(x)  # Output: 10

print(x)  # Output: 10
```

✓Corrección:

```
x = 5

if True:

    new_x = 10  # Nombre distinto

    print(new_x)  # Output: 10

print(x)  # Output: 5
```

5. Pasar por alto las excepciones (ZeroDivisionError)

Si no manejas excepciones, tu programa puede **detenerse inesperadamente**.

✗Error:

```
x = 10
```

```
y = 0
```

```
result = x / y # ZeroDivisionError
```

❑ Error:

```
ZeroDivisionError: division by zero
```

✓ **Corrección:**

```
x = 10
```

```
y = 0
```

```
try:
```

```
    result = x / y
```

```
except ZeroDivisionError:
```

```
    print("Error: Division by zero")
```

✓ Output:

```
Error: Division by zero
```

Conclusión

Los errores más comunes en Python y cómo evitarlos:

1. **Indentación** → Mantén sangría consistente.
2. **Ámbito** → Declara variables en el lugar correcto.
3. **Inmutabilidad** → Recuerda que strings no se modifican, se crean nuevos.
4. **Reasignación accidental** → Usa nombres claros y únicos.

5. **Excepciones** → Usa `try-except` para evitar bloqueos.

Más allá de lo básico: Errores avanzados y buenas prácticas en Python

Además de los errores comunes, existen problemas más avanzados que conviene entender para escribir programas **eficientes, estables y fáciles de mantener**.

1. Importaciones circulares

En Python, puedes importar código de otros archivos o librerías.

El problema aparece cuando **dos módulos dependen uno del otro**, generando un bucle.

Ejemplo:

✗Error:

```
# archivo1.py
```

```
import archivo2
```

```
def funcion1():
```

```
    return "Función en archivo1"
```

```
# archivo2.py
```

```
import archivo1
```

```
def funcion2():
```

```
    return "Función en archivo2"
```

Esto crea una **importación circular** (archivo1 necesita archivo2 y viceversa).
Resultado → Python no sabe por dónde empezar y puede lanzar errores como:

```
ImportError: cannot import name 'funcion1' from partially  
initialized module 'archivo1'
```

Soluciones:

- Reorganizar el código para evitar dependencias mutuas.
- Usar **importaciones locales** dentro de funciones en lugar de globales.
- Extraer el código común a un **tercer módulo** y que ambos lo importen.

2. Administración de la memoria

Un código ineficiente puede consumir recursos hasta colapsar el sistema.

✗Error:

```
my_string = ""  
  
lots_of_as = "a" * 1000000000  
  
while True:  
  
    my_string = my_string + lots_of_as # Bucle infinito -> agota la  
memoria
```

☐ Esto crea cadenas gigantes hasta que el ordenador **se queda sin memoria**.

Buenas prácticas:

- Usar **estructuras más eficientes** como listas o generadores.
- Evitar bucles infinitos con concatenaciones enormes.
- Procesar datos por **bloques** en lugar de todo a la vez.

✓ Ejemplo eficiente:

```
# Uso de join en lugar de concatenación infinita

lista = ["a" * 100 for _ in range(1000)]

resultado = "".join(lista)

print(len(resultado))
```

3. Pruebas y depuración

Los programas grandes son como rascacielos: difíciles de revisar manualmente. Por eso, se usan **pruebas automatizadas** y herramientas de depuración.

Herramientas de depuración:

- `print()` para rastreo simple.
- `pdb` (Python Debugger) para depuración paso a paso.
- IDEs como PyCharm o VS Code con depuradores visuales.

Buenas prácticas en pruebas:

- Usar **pruebas unitarias** con `unittest` o `pytest`.
- Probar **casos extremos** (ej: dividir por cero, listas vacías).
- Implementar **pruebas automatizadas** en proyectos grandes.

✓ Ejemplo de prueba unitaria:

```
import unittest
```

```
def suma(a, b):
```

```
    return a + b
```

```
class TestSuma(unittest.TestCase):
```

```
def test_suma(self):  
  
    self.assertEqual(suma(2, 3), 5)  
  
    self.assertEqual(suma(-1, 1), 0)  
  
  
if __name__ == '__main__':  
  
    unittest.main()
```

Conclusión final

Los errores avanzados que debes vigilar son:

1. **Importaciones circulares** → organiza tu código modularmente.
2. **Mala administración de memoria** → usa estructuras eficientes y evita bucles infinitos.
3. **Falta de pruebas y depuración** → adopta pruebas automatizadas y depuradores.

Con práctica y buenas prácticas, escribirás programas **robustos, escalables y confiables**.

Actividad: Variables y bucles

1. **Pregunta 1**
Paso 1: Almacenar los datos de temperatura

Instrucciones:

Necesita dos listas para almacenar los datos de temperatura.

La primera lista, `celsius_temperatures`, ya ha sido creada.

Añada a ella las siguientes temperaturas Celsius de muestra: `0, 10, 25, 32, 100` a `celsius_temperatures`.

Cree la segunda lista, `fahrenheit_temperatures`, como una lista vacía para almacenar las temperaturas Fahrenheit convertidas más tarde. En este momento debería ser una lista vacía.

Imprime ambas listas para observar su estado inicial.

```
# Create a list and add sample Celsius temperatures to it
celsius_temperatures = [] # Start with an empty list (do not modify)

# Add the Celsius temperatures to the list
celsius_temperatures = [0, 10, 25, 32, 100]

# Create an empty list to store Fahrenheit temperatures
fahrenheit_temperatures = []

# Print both lists (do not modify)
```

Restablecer

Celsius Temperatures: `[0, 10, 25, 32, 100]`

Fahrenheit Temperatures: `[]`

1 punto

2. Pregunta 2

Paso 2: Convertir temperaturas

Instrucciones:

Es hora de convertir las temperaturas Celsius a Fahrenheit:

Se le ha proporcionado un bucle `for` para iterar a través de cada valor `celsius` en la lista `celsius_temperatures`.

Dentro del bucle:

- Aplica la fórmula de conversión de Celsius a Fahrenheit:
 $F = (C * 9/5) + 32$, donde **F** es la temperatura Fahrenheit y **C** es la temperatura Celsius.
- El código para almacenar cada valor calculado `fahrenheit` en la lista `fahrenheit_temperatures` utilizando el método `append()` ha sido proporcionado para usted.

Imprima los resultados.

```
# Lists from Step 1 (do not modify)
celsius_temperatures = [0, 10, 25, 32, 100]
fahrenheit_temperatures = []

# Convert each Celsius temperature to Fahrenheit
for celsius in celsius_temperatures: # Start the loop (do not
    modify this line)
    fahrenheit = (celsius * 9/5) + 32

    fahrenheit_temperatures.append(fahrenheit) # Append to the list
(do not modify this line)
```

Restablecer

```
Celsius Temperatures: [0, 10, 25, 32, 100]
Fahrenheit Temperatures: [32.0, 50.0, 77.0, 89.6, 212.0]
```

1 punto

Escenario: *Domando los datos de temperatura con Python*

Eres un Analista de datos trabajando en un proyecto meteorológico. Ha recopilado una serie de lecturas de temperatura en grados Celsius, pero sus herramientas de análisis e informes requieren temperaturas en grados Fahrenheit.

Es hora de poner en práctica tus conocimientos de Python. Crearás un programa que convierte eficientemente una lista de temperaturas Celsius en sus equivalentes Fahrenheit, demostrando el poder de los bucles y variables en el manejo de transformaciones de datos del mundo real.

Resumen de la actividad:

- Crear e inicializar listas para almacenar datos de temperatura.
- Utiliza un bucle para iterar a través de las temperaturas Celsius.
- Aplica la fórmula de conversión de Celsius a Fahrenheit.
- Almacena las temperaturas Fahrenheit convertidas en una nueva lista.

Cuestionario de sentencias y bucles

1. Estás escribiendo un programa en Python para determinar si un cliente tiene derecho a un descuento en función del importe de su compra. ¿Cuál de las siguientes sentencias de decisión sería la más adecuada para esta tarea en Python? Seleccione la mejor respuesta.

- ☐ `print` declaración
- ☐ `while` bucle
- ☐ `for` bucle
- ☒ `if-else` declaración

✓ **Correcto**

Correcto Una sentencia `if-else` permite ejecutar diferentes bloques de código en Python en función de si el importe de la compra del cliente cumple los criterios de descuento.

2. Estás construyendo un juego de adivinar números en Python. El jugador tiene que adivinar un número secreto, y el programa proporciona información si el número adivinado es demasiado alto o demasiado bajo. ¿Cómo podrían ayudarte los bucles y las sentencias condicionales de Python a implementar la lógica de este juego? Selecciona la mejor respuesta.

- ☐ Un bucle `while` podría generar un número aleatorio utilizando el módulo `random`, y una sentencia `if` podría asignar puntos al jugador.
- ☒ Un bucle `while` podría preguntar repetidamente al jugador por su suposición utilizando `input()`, y una sentencia `if-elif-else` podría comprobar si la suposición es correcta, demasiado alta o demasiado baja.
- ☐ Un bucle `for` podría realizar un seguimiento de la puntuación del jugador, y una sentencia `if-else` podría proporcionar pistas.
- ☐ Un bucle `for` podría mostrar las instrucciones del juego, y una sentencia `if` podría terminar el juego.

✓ **Correcto**

Correcto Esto describe correctamente cómo el bucle `while` de Python y las sentencias `if-elif-else` se pueden utilizar para crear el bucle de juego principal.

3. Estás creando un programa para determinar si un estudiante puede optar a una beca en función de su nota media. ¿Qué Sentencia condicional utilizarías para comprobar si el GPA del estudiante está por encima de un determinado umbral? Seleccione la mejor respuesta.

- ☐ `else`
- ☒ `if`
- ☐ `elif`
- ☐ `for`

✔ **Correcto**

Correcto La sentencia `if` es la base de la lógica condicional, ya que permite comprobar una condición específica y ejecutar código si es cierta.

4. Estás creando un sencillo juego de adivinanzas en el que el jugador tiene que adivinar un número secreto. Tienen un número ilimitado de intentos, pero sólo hay un juego. Después de adivinar la respuesta correcta, el programa termina. ¿Cuál de las siguientes tareas dentro de esa única ronda requeriría repetir una acción (usando un bucle)? Seleccione todas las que correspondan.

- ☐ Generar la respuesta correcta al inicio del juego.
- ☐ Pedir al jugador que adivine varias veces en una misma ronda.
- ☒ Mostrar un mensaje indicando al usuario que ha acertado y mostrando el número de aciertos.

✘ **Esto no debería estar seleccionado**

No del todo. Debería mostrarse una vez durante la ejecución del programa, después de que el usuario haya adivinado correctamente.

- ☒ Comprobar si la suposición del jugador coincide con la respuesta correcta.

✔ **Correcto**

Correcto ASi el usuario tiene múltiples intentos, cada intento debe ser comparado con la respuesta correcta.

5. Un equipo de desarrollo de software está creando una aplicación de planificación financiera. Quieren incorporar funciones que ayuden a los usuarios a realizar un seguimiento de los gastos, establecer presupuestos y recibir alertas de gastos inusuales. ¿Cuál de los siguientes aspectos de esta aplicación dependería en gran medida del uso de sentencias condicionales y bucles en Python? Seleccione todo lo que corresponda.

☒ Envío de una notificación si el usuario supera su presupuesto preestablecido para una categoría específica

☒ **Correcto**

Correcto Se trataría de una sentencia condicional para comprobar si el gasto actual en una categoría es superior al importe presupuestado.

☒ Categorizar los gastos en diferentes tipos (por ejemplo, "Comida", "Vivienda", "Ocio")

☒ **Correcto**

Correcto Sentencia condicional podría utilizarse para comprobar la descripción de un gasto y asignarlo a la categoría apropiada.

☒ Permite a los usuarios establecer distintos límites presupuestarios para cada mes.

☒ **Correcto**

Correcto Se podrían utilizar bucles para iterar a través de los meses y aplicar los límites presupuestarios correspondientes, potencialmente con sentencias condicionales para manejar ajustes o casos especiales.

☐ Generar un informe visual que muestre los patrones de gasto del usuario a lo largo del tiempo

6. Está depurando un programa Python que calcula el precio final de un artículo después de aplicar los descuentos y el impuesto sobre las ventas. El fragmento de código es el siguiente:

```
original_price = 100
discount_percentage = 20
sales_tax_rate = 0.08

discount_amount = original_price * (discount_percentage / 100)
discounted_price = original_price - discount_amount
sales_tax = discounted_price * sales_tax_rate
final_price = discounted_price + sales_tax

print(final_price)
```

¿En qué orden Python ejecutará las líneas de código para determinar el `final_price`? Seleccione la mejor respuesta.

- ☒ Calculará el `discount_amount`, luego el `discounted_price`, seguido del `sales_tax`, y por último el `final_price`.
- ☐ Calculará el `discount_amount`, luego el `sales_tax`, luego el `discounted_price`, y finalmente el `final_price`.
- ☐ Calculará primero el `discounted_price`, luego el `discount_amount`, seguido del `sales_tax`, y por último el `final_price`.
- ☐ Primero calculará el `final_price` utilizando directamente el `original_price`, y después calculará los demás valores para su visualización.

☒ **Correcto**

Correcto Python ejecuta el código línea por línea, respetando el orden de las operaciones y las dependencias entre variables.

7. Un desarrollador junior está trabajando en un programa Python. Este programa no se ejecuta y muestra un error sobre una variable que no está definida. Al mirar el programa, la variable está definida. ¿Cuál de los siguientes conceptos está más probablemente relacionado con este error? Seleccione la mejor respuesta.

- ☐ Error de administración de la memoria: Se está agotando la memoria.
- ☐ Sombreado de nombres: Utilizar el mismo nombre de variable para diferentes propósitos dentro del programa.
- ☒ Ámbito de la variable: Intentar acceder a una variable fuera de su ámbito definido.
- ☐ Pasar por alto excepciones: El programa no está manejando correctamente una excepción.

☒ **Correcto**

Correcto Si la variable está definida pero aparece un error que indica que no está definida, se trata de un error de ámbito común.

Introducción a las Listas en Python

- **Definición:**
Las listas son estructuras de datos fundamentales y versátiles en Python que pueden contener múltiples tipos de elementos (números, cadenas, booleanos e incluso otras listas).
- **Ventaja principal:**
A diferencia de las matrices en otros lenguajes, **las listas de Python no restringen el tipo de datos**, lo que las convierte en una herramienta flexible para organización de datos y algoritmos.

Declaración de listas – Ejemplos comunes:

Lista vacía: inicializada para llenarse durante la ejecución.

```
new_list = []
```

Números primos: almacenar enteros menores o iguales a 13.

```
primes = [2, 3, 5, 7, 11, 13]
```

Vocales: lista de cadenas con vocales.

```
vowels = ["a", "e", "i", "o", "u"]
```

Resultados de lanzamientos de moneda (booleanos):

```
coin_flips = [True, False, True, True, False]
```

Valores mixtos (ejemplo de información de un estudiante):

```
student_record = ["Aashvi", 24, "Computer Science", 3.8, True]
```

Introducción y Manipulación de Listas en Python

¿Qué son las listas?

- Son estructuras de datos fundamentales y muy versátiles en Python.
- Pueden contener diferentes tipos de elementos (números, cadenas, booleanos e incluso otras listas).
- Se utilizan para organizar datos y facilitar la implementación de algoritmos.

Creación y visualización de listas

Ejemplo de lista de la compra:

```
grocery_list = ["milk", "hummus", "bread", "cheese", "apples"]
```

Imprimir lista completa:

```
print(grocery_list)  
# ['milk', 'hummus', 'bread', 'cheese', 'apples']
```

-

Indexación

- Cada elemento tiene un **índice** que empieza en 0.
- Ejemplo: "milk" está en el índice 0.

Índice	VALUE
0	leche
1	hummus
2	pan
3	queso
4	manzanas

Para imprimir un elemento específico:

```
print(grocery_list[0]) # milk
```

-

Longitud de la lista:

```
print(len(grocery_list)) # 5
```

-

Último elemento:

```
print(grocery_list[len(grocery_list)-1])  
# o más simple:  
last_index = len(grocery_list) - 1  
print(grocery_list[last_index])
```

-

Rebanado (slicing)

Permite acceder a una parte de la lista.

```
print(grocery_list[0:2]) # ['milk', 'hummus']
```

Devuelve los elementos desde el índice inicial hasta el índice final (sin incluirlo).

Comprensión de listas

Forma **concisa y eficiente** de crear nuevas listas a partir de otras.

Sintaxis:

```
[expression for item in iterable]
```

Ejemplo – aumentar 10 puntos a cada nota:

```
exam_scores = [55, 70, 78, 52, 68]  
curve_amount = 10  
curved_grades = [score + curve_amount for score in exam_scores]  
print("Curved scores:", curved_grades)  
# [65, 80, 88, 62, 78]
```

Errores comunes con listas

- **IndexError**: acceder a un índice inexistente.
- **ValueError**: intentar eliminar un valor que no está en la lista.

Listas bidimensionales (listas dentro de listas)

Ejemplo de cuadrícula:

```
grid = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
print(grid[1][2])  # 6
```

Operaciones comunes con listas

Dada la lista:

```
grocery_list = ["milk", "hummus", "bread", "cheese", "apples"]
```

- `len(lista)` → número de elementos.
- `list.append(x)` → añade al final.
- `list.insert(i, x)` → inserta en una posición específica.
- `list.remove(x)` → elimina la primera aparición de `x`.
- `list.sort()` → ordena la lista.
- `list.reverse()` → invierte el orden.
- `list.count(x)` → cuenta cuántas veces aparece `x`.
- `x in lista` → comprueba si un valor está en la lista.

- `list.index(x)` → devuelve la posición de la primera aparición de `x`.

Operaciones comunes con listas

Dada la lista: recuerda que el primer elemento es 0

```
grocery_list = ["milk", "hummus", "bread", "cheese", "apples"]
```

Lista original

```
grocery_list = ["milk", "hummus", "bread", "cheese", "apples"]  
print("Lista original:", grocery_list)
```

1. `len(lista)`: número de elementos

```
print("Número de elementos:", len(grocery_list))
```

2. `append(x)`: añadir al final

```
grocery_list.append("bananas")  
print("Después de append('bananas'):", grocery_list)
```

3. `insert(i, x)`: insertar en un índice específico

```
grocery_list.insert(1, "yogurt")  
print("Después de insert(1, 'yogurt'):", grocery_list)
```

4. `remove(x)`: eliminar la primera aparición de un elemento

```
grocery_list.remove("milk")  
print("Después de remove('milk'):", grocery_list)
```

5. `sort()`: ordenar alfabéticamente

```
grocery_list.sort()  
print("Después de sort():", grocery_list)
```

6. `reverse()`: invertir el orden

```
grocery_list.reverse()  
print("Después de reverse():", grocery_list)
```

7. `count(x)`: contar cuántas veces aparece un elemento

```
print("Veces que aparece 'apples':", grocery_list.count("apples"))
```

SALIDAS:

Lista original: ['milk', 'hummus', 'bread', 'cheese', 'apples']

Número de elementos: 5

Después de `append('bananas')`: ['milk', 'hummus', 'bread', 'cheese', 'apples', 'bananas']
Después de `insert(1, 'yogurt')`: ['milk', 'yogurt', 'hummus', 'bread', 'cheese', 'apples', 'bananas']
Después de `remove('milk')`: ['yogurt', 'hummus', 'bread', 'cheese', 'apples', 'bananas']
Después de `sort()`: ['apples', 'bananas', 'bread', 'cheese', 'hummus', 'yogurt']
Después de `reverse()`: ['yogurt', 'hummus', 'cheese', 'bread', 'bananas', 'apples']
Veces que aparece 'apples': 1

Conclusión:

Las listas son una de las estructuras más poderosas y usadas en Python. Permiten almacenar datos, acceder a ellos mediante índices, rebanarlos, transformarlos con comprensiones, e incluso organizarlos con operaciones avanzadas como ordenamiento o conteo.

Listas en Python:

Las **listas** en Python son estructuras de datos que permiten almacenar, organizar y manipular colecciones de elementos de manera eficiente. Funcionan como contenedores que mantienen el **orden de los elementos**, son **mutables** (pueden modificarse, agregarse o eliminarse elementos) y son **versátiles**, ya que pueden contener distintos tipos de datos, incluidos números, cadenas, booleanos o incluso otras listas.

Atributos clave de las listas:

- **Almacenamiento organizado:** mantienen el orden de los datos y facilitan el acceso o la iteración.
- **Mutabilidad:** se pueden modificar tras su creación, ideal para datos que cambian con el tiempo.
- **Versatilidad:** permiten almacenar múltiples tipos de datos y estructuras complejas.

Aplicaciones en el mundo real:

- **Comercio electrónico:** gestionar catálogos de productos, precios, inventario y ventas.
- **Redes sociales:** almacenar perfiles, publicaciones, comentarios y preferencias de usuario.
- **Salud:** manejar historiales médicos, medicamentos y resultados de pruebas de pacientes.

Operaciones esenciales con listas:

1. **Append (anexar):** añade elementos al final de la lista.
2. **Indexing (indexar):** acceder a elementos por su posición usando índices (empiezan en 0).
3. **Slicing (segmentación):** extraer subconjuntos de la lista usando rangos de índices.

Las listas son fundamentales para organizar datos en Python, y dominar sus operaciones básicas permite gestionar información de manera eficiente y flexible, con posibilidades de avanzar a técnicas más complejas a medida que se adquiere experiencia.

Manipulación de listas en Python:

Las **manipulaciones de listas** en Python permiten **transformar, filtrar y organizar datos** de manera eficiente, haciendo que el código sea más claro y ahorrando tiempo al trabajar con conjuntos de datos complejos. Estas técnicas son útiles para limpiar datos, eliminar duplicados, filtrar información específica y realizar cálculos o análisis.

Aplicaciones prácticas:

- **Limpieza de datos:** corregir errores tipográficos, eliminar espacios o valores incompletos en listas de clientes.
- **Análisis de datos:** calcular promedios, identificar tendencias o filtrar resultados de experimentos.
- **Automatización:** renombrar archivos, personalizar correos electrónicos o agregar información automáticamente.

Técnicas comunes de manipulación de listas:

1. **Slicing (rebanado):** extraer partes específicas de una lista según posiciones o rangos, útil para enfocarse en subconjuntos de datos.
2. **List comprehension (comprensión de listas):** crear nuevas listas filtrando, transformando o calculando valores a partir de listas existentes, todo en una sola línea de código.
3. **Funciones integradas:**

- `append()`: añadir un elemento al final.
- `insert()`: insertar un elemento en una posición específica.
- `pop()`: eliminar y devolver un elemento por índice.
- `remove()`: eliminar la primera aparición de un valor.
- `sort()`: ordenar elementos ascendente o descendentemente.
- `reverse()`: invertir el orden de los elementos.
- `count()`: contar cuántas veces aparece un valor.
- `index()`: devolver la posición de la primera aparición de un valor.

Dominar estas técnicas permite **analizar datos, automatizar tareas repetitivas y crear aplicaciones más eficientes** usando listas en Python.

Pregunta

Está trabajando con una lista de datos de clientes que incluye nombres, direcciones de correo electrónico e historial de compras. Necesita identificar y extraer los clientes que han realizado compras superiores a un importe determinado. ¿Qué técnica de manipulación de listas sería la MÁS eficaz para conseguirlo? Seleccione la mejor respuesta.

- ☒ Utilice una comprensión de lista para crear una nueva lista que contenga sólo los clientes cuyo historial de compras supere el umbral.
- ☐ Recorrer manualmente la lista mediante un bucle `for`, comprobando el historial de compras de cada cliente y añadiéndolo a una nueva lista si cumple los criterios.
- ☐ Utilice el método `append()` para añadir clientes de alto gasto a una nueva lista.
- ☐ Utilice el método `sort()` para ordenar a los clientes por importe de compra y, a continuación, utilice el corte para extraer los que más gastan.

☒ **Correcto**

Correcto Las comprensiones de listas están diseñadas para filtrar y transformar listas de forma concisa y eficiente, por lo que son ideales para esta tarea.

Actividad lista de la compra (grocery_list.csv)

1. ¿Qué biblioteca de Python se utiliza principalmente para la exploración y manipulación de datos en este proyecto? Seleccione la mejor respuesta.

- ☐ Matplotlib
- ☐ Seaborn
- ☒ pandas
- ☐ NumPy

✓ **Correcto**

Pandas es la biblioteca de referencia para cargar, explorar y manipular datos en Python.

2. Paso 2: Añadir artículos a la lista

Ahora, imaginemos que estás en casa añadiendo elementos a tu lista. Simularemos esto añadiendo un par de elementos directamente a tu lista de Python. Para ello, utilizarás una acción especial llamada **método** `append()`. Los métodos son como funciones integradas que realizan tareas específicas. En este caso, `append()` añade elementos al final de la lista, como si los anotaras al final de una lista de papel.

- Utilice el método `append()` para añadir "Kiwis" y "Frambuesas" a la lista `items_to_add` y ejecute la celda.
- Introduce el artículo que desees cuando se te pida, que también se añadirá a la lista.
- Observe la lista de la compra actualizada con los artículos recién añadidos impresos en la consola.

¿Qué método se utiliza para añadir nuevos artículos al final de la lista de la compra?

- ☐ `extend()`
- ☒ `append()`
- ☐ `add()`
- ☐ `insert()`

✓ **Correcto**

Correcto: `append()` es la forma estándar de añadir un elemento al final de una lista

3. Paso 3: Eliminar artículos de la lista

Vaya, parece que ya tiene algunos artículos en casa. No se preocupe Aprenderá a eliminar artículos de su lista utilizando otro práctico método llamado `remove()`. Esto es esencial para mantener su lista exacta y evitar compras innecesarias.

- Utilice el método `remove()` para eliminar "Huevos" y "Manzanas" de `grocery_list` y ejecute la celda.

¿Qué método está diseñado específicamente para eliminar elementos de una lista por su valor?

- ☐ `pop()`
- ☒ `remove()`
- ☐ `delete()`
- ☐ `clear()`

✓ **Correcto**

Correcto: `remove()` busca y elimina la primera aparición de un valor especificado

4. ¿Qué tipo de error se produce si se intenta eliminar un elemento que no existe en la lista utilizando el método `remove()`?

- ☐ `IndexError`
- ☒ `ValueError`
- ☐ `TypeError`
- ☐ `KeyError`

✓ **Correcto**

Correcto! `remove()` lanza un `ValueError` si el valor especificado no se encuentra en la lista

Cuestionario organizar tus datos

1. Estás trabajando en un proyecto de Python en el que necesitas almacenar y manipular una colección de puntuaciones de un juego. Has decidido utilizar una lista para almacenar estas puntuaciones. ¿Cuál de las siguientes afirmaciones describe correctamente las listas y sus operaciones en Python? Selecciona todas las que correspondan.

- ☐ El método `append()` añade un elemento al principio de la lista, mientras que el método `remove()` elimina el último elemento.
- ☐ Se produce un error en `ValueError` cuando se intenta acceder a un elemento utilizando un índice no válido, como un número negativo o un índice superior a la longitud de la lista.
- ☒ Una lista es una colección ordenada y mutable de elementos, que permite almacenar diferentes tipos de datos dentro de la misma lista y modificar sus elementos después de la creación.

✓ **Correcto**

Correcto Las listas son ordenadas, lo que significa que los elementos mantienen su orden de inserción, y mutables, lo que permite modificaciones. También pueden contener tipos de datos mixtos.

- ☒ Las listas son esenciales en Python para organizar datos, permitiendo realizar operaciones como ordenar, buscar e iterar a través de elementos.

✓ **Correcto**

Correcto Las listas proporcionan una estructura flexible para la organización y manipulación de datos, permitiendo diversas operaciones para un manejo eficiente de los datos.

2. Estás desarrollando un programa para simular la cola de clientes de un restaurante popular. Necesitas almacenar los nombres de los clientes en el orden en que llegan. ¿Qué característica clave de las listas de Python las hace adecuadas para esta tarea? Selecciona la mejor respuesta.

- ☐ Mutabilidad, ya que puede añadir y eliminar clientes de la cola a medida que llegan y se van.
- ☒ Orden, ya que garantiza que se pueda acceder y atender a los clientes en el orden en que fueron añadidos a la lista.
- ☐ Versatilidad, ya que puede almacenar en la lista varios tipos de datos, como nombres y horas de llegada.
- ☐ La posibilidad de añadir elementos, ya que permite añadir nuevos clientes a la cola.

✓ **Correcto**

Correcto Las listas conservan el orden de los elementos, lo que es crucial para gestionar una cola por orden de llegada.

3. Una empresa de Redes sociales necesita almacenar varios tipos de datos para cada perfil de usuario, incluyendo su nombre de usuario (texto), edad (número), una lista de sus amigos (otra lista), y si están actualmente en línea (booleano). ¿Qué característica de las listas de Python las hace adecuadas para gestionar esta información tan diversa sobre los usuarios? Selecciona la mejor respuesta.

- ☐ Mutabilidad, ya que permite actualizar el perfil si el usuario cambia de edad o añade un amigo.
- ☒ Versatilidad, ya que permite almacenar distintos tipos de datos dentro de una misma lista.
- ☐ Indexación, ya que permite acceder a información específica del perfil, como la edad del usuario.
- ☐ Orden, ya que garantiza que los datos del usuario se almacenan en una secuencia predecible.

✓ **Correcto**

Correcto Las listas pueden alojar varios tipos de datos, lo que resulta esencial para almacenar la diversa información de un perfil de usuario.

4. Está trabajando con un conjunto de datos de opiniones de clientes, en el que cada entrada contiene una cadena de texto. Necesita limpiar estos datos eliminando los espacios sobrantes y convirtiendo todo el texto a minúsculas para que el análisis sea coherente. ¿Qué técnica de manipulación de listas sería la más adecuada? Seleccione la mejor respuesta.

- ☐ `sort()` método: Ordene las entradas de comentarios alfabéticamente para identificar incoherencias en el uso de mayúsculas.
- ☒ Comprensión de listas: Crear una nueva lista con cadenas de texto depuradas, aplicando las transformaciones necesarias a cada elemento.
- ☐ `index()` método: Buscar la posición de las entradas con espacios de más o mayúsculas incoherentes para su posterior tratamiento.
- ☐ `pop()` método: Elimine las entradas con espacios de más o mayúsculas incoherentes.

✓ **Correcto**

Correcto Las comprensiones de listas permiten crear de forma eficiente una nueva lista aplicando a cada elemento de la lista original operaciones como la eliminación de los espacios sobrantes y la conversión a minúsculas.

5. Un Analista de datos necesita procesar un gran conjunto de datos de reseñas de productos. Muchas de ellas contienen información irrelevante, como caracteres especiales y etiquetas HTML. ¿Qué concepto de Python sería más útil para automatizar la tarea de limpiar estas reseñas y extraer sólo la información textual esencial? Selecciona la mejor respuesta.

- ☐ Indexación: Acceda a caracteres específicos dentro de cada reseña para identificar y eliminar información irrelevante
- ☐ Rebanar: Extraer partes de las reseñas que contengan comentarios relevantes
- ☒ Comprensiones de lista combinadas con métodos de cadena: Iterar por las revisiones, eliminar los caracteres no deseados y crear una nueva lista con el texto depurado
- ☐ Métodos de lista incorporados como `append()` y `insert()`: Añade reseñas depuradas a una nueva lista

✓ **Correcto**

Correcto Este enfoque combina la eficacia de las comprensiones de listas para iterar y filtrar con la potencia de los métodos de cadenas para manipular el texto dentro de cada revisión.

ACTIVIDAD conceptos básicos de Python

1. Un desarrollador de software está diseñando un programa para el sistema en línea de una biblioteca. El programa debe determinar si un libro está vencido y calcular los cargos por mora según el número de días de atraso. ¿Qué Sentencia condicional sería la más adecuada para esta tarea? Seleccione la mejor respuesta.

- ☐ Una sentencia `if` para tratar el caso en que el libro no esté vencido
- ☒ Una sentencia `if` para comprobar si la fecha de vencimiento es anterior a la fecha actual
- ☐ Una combinación de `if` y `else` para cubrir tanto los casos de impago como de impago
- ☐ Una sentencia `elif` para comprobar si el libro se ha retrasado un número determinado de días

✓ **Correcto**

Correcto Una sentencia `if` es ideal para comprobar si se cumple una condición (libro vencido).

2. Usted está desarrollando un juego en el que el jugador tiene que adivinar un número secreto entre 1 y 100. ¿Qué bucle sería el más adecuado para preguntar repetidamente al jugador hasta que adivine el número correcto? Seleccione la mejor respuesta.

- ☐ Un bucle `for`, ya que está diseñado para iterar sobre una secuencia conocida de números
- ☐ En este caso podrían utilizarse indistintamente los bucles `for` o `while`
- ☒ Un bucle `while`, ya que continúa ejecutándose mientras la suposición del jugador sea incorrecta
- ☐ Bucles anidados `for`, para recorrer todos los números posibles y compararlos con la suposición del jugador

✓ **Correcto**

Correcto Un bucle `while` es ideal aquí porque el número de adivinanzas es desconocido de antemano, y el bucle necesita continuar hasta que el jugador adivine correctamente.

3. Está escribiendo un programa para analizar un conjunto de datos de calificaciones de estudiantes. Después de calcular las calificaciones, desea mostrar "Aprobado" o "Suspendido" para un estudiante individual. ¿Qué mecanismo de flujo de control sería el más adecuado para realizar esta determinación? Seleccione la mejor respuesta.

- ☐ Un bucle `while` para calcular continuamente promedios hasta que se cumpla una condición específica
- ☒ Sentencia condicional (`if, else`) para determinar si la nota media de un alumno está por encima o por debajo del umbral de aprobado
- ☐ Bucles anidados `while` para calcular promedios y comprobar el estado de aprobado/reprobado simultáneamente
- ☐ Un bucle `for` para recorrer las calificaciones de cada alumno y calcular su media

✓ **Correcto**

Correcto Puede utilizar una sentencia if con una condición else para determinar si un alumno aprueba o suspende.

4. Mientras depuras tu programa Python, sospechas que hay un problema con la forma en que se llaman las funciones y sus valores de retorno. ¿Qué herramienta o técnica sería más útil para comprender la secuencia de llamadas a funciones e identificar posibles problemas? Seleccione la mejor respuesta.

- ☐ Lectura del código línea por línea
- ☐ Comprobación de la salida del programa en busca de mensajes de error
- ☒ Examinar la pila de llamadas
- ☐ Consulta de la documentación del programa

✓ **Correcto**

Correcto La pila de llamadas proporciona una instantánea de las llamadas a funciones activas

5. Estás escribiendo un programa en Python que contiene un bucle while. Cada vez que pase por el bucle, el programa añadirá un valor a una variable de cadena. ¿Qué error de ejecución común es más probable que se produzca? Selecciona la mejor respuesta.

- ☐ Cuestiones de alcance
- ☐ Importación circular
- ☒ Inestabilidad del sistema por problemas de memoria
- ☐ Seguimiento de nombres

✓ **Correcto**

Correcto Añadir a una variable de forma continua puede conducir a un uso excesivo de memoria, lo que puede provocar inestabilidad y fallos.

6. Un desarrollador de software tiene la tarea de crear un programa para almacenar una secuencia de imágenes para una animación. ¿Qué característica de las listas en Python sería más útil para garantizar que las imágenes se muestren en el orden correcto? Seleccione la mejor respuesta.

- ☐ Indexación
- ☐ Versatilidad
- ☒ ORDER BY
- ☐ Mutabilidad

✓ **Correcto**

¡Correcto! Las listas en Python mantienen el orden en el que se añaden los elementos, lo que es crucial para asegurar que los fotogramas de la animación se muestran en la secuencia correcta.

7. Un servicio de streaming de música utiliza listas para almacenar listas de reproducción. ¿Qué operación de lista utilizaría un usuario para añadir una nueva canción al final de su lista de reproducción? Seleccione la mejor respuesta.

- ☐ Rebanar
- ☐ Indexación
- ☐ Ordenación
- ☒ Añadiendo

✔ **Correcto**

Correcto Añadir añade la nueva canción al final de la lista de reproducción, ampliando la lista.

8. Un científico de datos está trabajando con un gran conjunto de datos de opiniones de clientes. ¿Qué operaciones de lista serían útiles para analizar secciones o características específicas de este conjunto de datos? Seleccione todas las que corresponda.

- ☐ Insertar
- ☐ Añadiendo
- ☒ Indexación

✔ **Correcto**

Correcto La indexación permite acceder a reseñas individuales para un examen detallado.

- ☒ Rebanar

✔ **Correcto**

Correcto La segmentación permite extraer subconjuntos de opiniones, centrándose quizás en periodos o categorías de productos específicos.

9. Un Analista de datos tiene una lista de cifras de ventas del último año, pero necesita analizar las tendencias específicamente del último trimestre. ¿Qué técnica extraería eficazmente este subconjunto de datos? Seleccione la mejor respuesta.

- ☐ `pop()`
- ☐ `insert()`
- ☐ Comprensión de la lista
- ☒ Rebanar

✓ **Correcto**

Correcto La segmentación permite extraer una parte específica de la lista, como las cifras de ventas del último trimestre.

10. Un ingeniero de software está desarrollando un programa para gestionar una cola de tareas. Las nuevas tareas se añaden al final de la cola y las tareas completadas se eliminan del principio. ¿Qué dos funciones de lista integradas serían las más adecuadas para implementar este sistema de gestión de tareas? Seleccione la mejor respuesta.

- ☐ `sort()` y `reverse()`
- ☐ `count()` y `index()`
- ☐ `insert()` y `remove()`
- ☒ `append()` y `pop()`

✓ **Correcto**

Correcto! `append()` añade nuevas tareas al final de la cola, y `pop()` puede eliminar tareas completadas desde el principio utilizando `pop(0)`, simulando el comportamiento FIFO (Primero en entrar, primero en salir) de una cola.